

DANMARKS TEKNISKE UNIVERSITET



Introduction to Energy Analytics (46040)

MEMBERS OF GROUP 22

Oskar John Bruun Smith

s225207

Jacob Theilgaard

s223463

Lucas Lojo

s215244

Albert Foss

s225180

April 16, 2024

In this report, all students contributed equally to all parts and are in agreement to its content.

Contents

0	Introduction	2
1	Task 1 - ARIMA Model	2
1.1	Day-ahead prediction for the testing dataset using ARIMA	2
1.1.1	Method 1	2
1.1.2	Method 2	6
1.2	Day-ahead prediction using an exogenous variable in SARIMA model	7
2	Task 2 - LSTM Model	10
2.1	Day-ahead prediction for the testing dataset (LSTM)	10
2.2	Day-ahead prediction including an exogenous variable (LSTMX)	11
3	Task 3 - Comparing the performance of the ARIMA and LSTM models	13
3.1	Comparing the performance of the SARIMAX and LSTMX models	13
3.2	Recommendation	14
4	Task 4 - Total profits for December 2023	15
4.1	Comparing potential profit scenarios, either by LSTM forecast or by perfect price knowledge	16
4.2	Investigating using persistence model prices to optimize battery the day ahead .	17
4.3	Key features to deliver a good profit-maximizing bidding strategy	17
5	Lessons learned	18
6	Conclusion	18

0 Introduction

This report examines predictive models for electricity prices and strategies for optimizing energy storage. It compares ARIMA and LSTM models, analyzes their forecasting accuracy, and explores profit maximization through battery optimization, offering insights into effective energy management practices.

1 Task 1 - ARIMA Model

In Task 1, we aim to develop an ARIMA model to predict electricity prices. We use a Seasonal ARIMA model and add exogenous variables to increase performance. The goal is to create an ARIMA model that most closely predicts spot price data on a day-ahead basis most efficiently. We have two methods of using an ARIMA model to forecast spotprice data. The first method is one we call a semi-manual method. One where we pick the first three parameters for a seasonal ARIMA and employ the seasonal ARIMA parameters that are generated with an automatic python function. The second method is where we only employ the automatic ARIMA modelling functions ARIMA and FourierFeaturizer. Exogenous variables are also used in the second method to further improve our model and ultimately the RMSE.

1.1 Day-ahead prediction for the testing dataset using ARIMA

We shall use no exogenous variables to establish a day-ahead prediction for our testing dataset, and we shall finally compare a naive persistence forecast (24-hour lag) with our ARIMA model by their RMSE values. In this task we will mainly walk through our method 1 and how we pick the parameters for our SARIMA model, and the results. We shall also touch upon method 2, as that is the method we shall continue using for task 1.2.

1.1.1 Method 1

We qualitatively pick the parameters for our ARIMA model, and build on top of this with the `auto.arima` model. We compare the RMSE value between our model and the naive persistence forecast. To develop a seasonal ARIMA model for day-ahead electricity price prediction (Task 1.1), we used the supplied importing and preprocessing code. We plotted the spotprice data, where we distinguish between the training and the testing period:

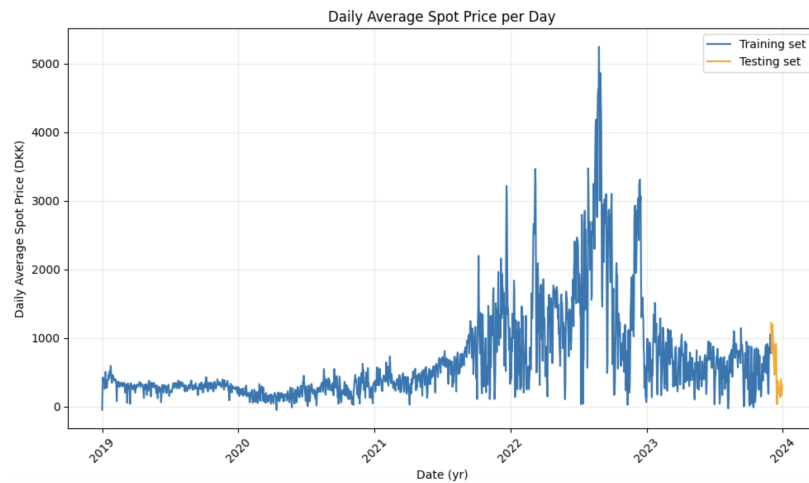


Figure 1: Daily average spot price per day

We can clearly see that the daily spotprice averages are not alike all throughout the testing data. We take this into consideration when picking the time-period for the data we work with. We would like our training data to look like the testing data. The reason for this is, we determine the coefficients in our model from the testing data. If the data has the same AR and MA patterns (and differencing for that matter) then our model will be able to predict future values better. So, we pick two months worth of spotprice data that is the closest to the prices that we want to predict as possible. So, we use the spotprice data for October and November 2023.

Before we move on to picking the parameters for our model, a quick introduction to the ARIMA model: The function of the ARIMA model is to employ differencing, autoregression, and a moving average to predict future values from historical data. The autoregressive part of the model, $AR(p)$, is a regression model that weights the significance of lagged values y until p -th time in the past as predictors. $I(d)$ stands for integrated, which tells us about how many time the difference is taken, until the original series becomes stationary. $MA(q)$ stand for moving average, which is a model that weights past forecast errors as predictors - regulating what to do if the forecast has been off in the past and how much to value each error in relation to how far in the past it is. A key aspect in picking an ARIMA model is to look at ACF and PACF plots. We have made the following two plots:

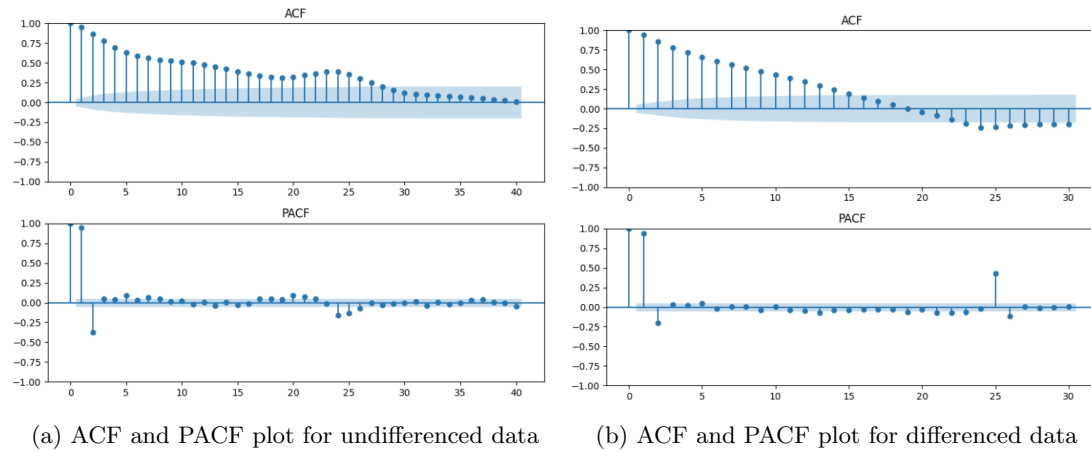


Figure 2: ACF and PACF plots for undifferenced and differenced data

For plot (a) we see that there is a local increase on the ACF plot at 12 hour lag, but especially at 24 hour lag. Furthermore, on the PACF plot we see a large increase, where the correlation becomes significant, around the 24 hour lag mark. The ACF measures autocorrelation, the correlation between a time series and its lagged values. The spikes indicates lags where there is significant autocorrelation between a value and its lagged values. The PACF finds the correlation between a time series and its lagged values, after we remove the effects of earlier lags. We can clearly see that there is a seasonal link at 24 hours between the time-series and its lagged series. So, we difference the data with a lag of 24 hours, and look at the ACF and PACF plots again (b). Usually, looking at ACF and PACF plots like that one would interpret it as being a $p = 2$ and $q = 0$ plot. We use AIC as a parameter to measure the effectiveness and precision of an ARIMA model, as our model must always take both the efficiency (we want as simple a model as possible) and precision (we want the best model as possible) into account. We have written code that computes the AIC for an arima model $\text{ARIMA}(2,1,16)(2,0,0)[24]$ all the way down to $\text{ARIMA}(2,1,1)(2,0,0)[24]$ to find the best model. We find $\text{ARIMA}(2,1,14)(2,0,0)[24]$. We found the seasonal parameters from the `auto_arima` function where we set `seasonal = true`. We evaluate our model by taking a closer look at the residuals:

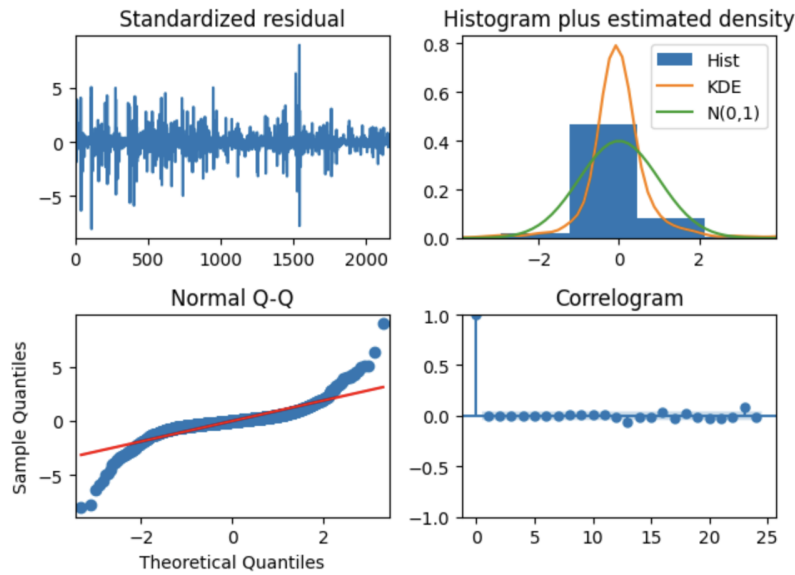


Figure 3: Enter Caption

We see that the residuals are not completely normally-distributed. The histogram indicates, that the mean of the residuals are 0 as hoped, but that our residuals have a smaller standard deviation than what we would have aimed for. This tells us that perhaps our model is very quick to eliminate differences between our function and the actual values as soon as they're known. Perhaps too many, or too high weighting in the MA model. The correlogram looks as it should. The Normal Q-Q plot indicates what I described - that our residuals are more tightly packed than what we would expect from a normal distribution. Our model is flawed, and it is not capturing all the patterns in the data. But, we employ it nonetheless. The following is a plot of the persistence forecast (24 h lag), forecast, and testing data:

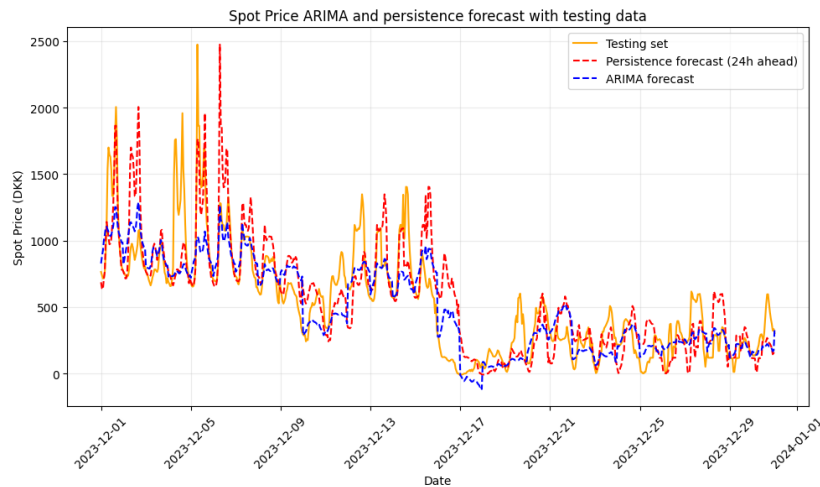


Figure 4: SARIMA forecast (method 1), persistence forecast, and testing data spotprice plot

As we can see our plot follows the general trend well. It doesn't follow the testing set into the extremes but rather stays quite close to the average of the time period, which fits well with the residual plot we found earlier. The RMSE values are 257 for the daily persistence and 232 for the SARIMA forecast.

Our SARIMA model performs better than the naive persistence, which is what we hoped. The RMSE is not too high compared to the fact that we have spot price values that are all the way up into the 2500's and a lot in the area 1000-1500. Though it isn't much better than the naive daily persistence, which we would have liked. Looking closer at the model it appears as if we are 24 hours ahead. Our model appears to be better than the naive persistence model, but it is clear that we are not getting all the information in our data.

1.1.2 Method 2

In method 2 we use automatic methods to find the parameters for the SARIMA model. We compare the RMSE for the results with the persistence data to see if the trouble of using an SARIMA model is worth it compared to a naive persistence forecast.

We use `auto.arima` with seasonality to find the best parameters within a specific amount of 'tries'. We fit the model to the data, and use it to forecast spotprice values on a day-ahead basis. We end up with the following plot:

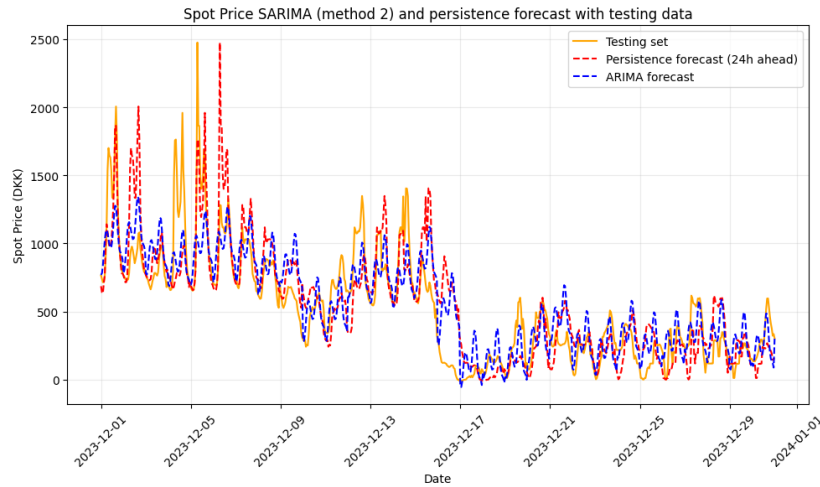


Figure 5: SARIMA forecast (method 2), persistence forecast, and testing data spotprice plot

The RMSE values are 257 for the persistence forecast and 327 for our forecast. We remark that this forecast is worse than the one we found manually. We could have let the `auto.arima` function try more permutations, and thereby have found a better fitting model. In this case the naive persistence model is better than our model, which means we have absolutely missed important aspects of our data, and even made a worse prediction than simply looking at data 24 hours ago. The ARIMA model is supposed to be better as we can look further into the past and weight the values depending on how long time ago it was, and even correct ourselves along the way by means of the moving average. By following this structured process, we aimed to develop a reliable seasonal ARIMA model for day-ahead electricity price prediction, facilitating informed decision-making in the energy market. Though, the automatic method produced a worse forecasting model than the naive persistence model. We shall try to improve the model in the next part of the report.

1.2 Day-ahead prediction using an exogenous variable in SARIMA model

We continue with method 2 and incorporate exogenous variables in our model. We would like to incorporate exogenous variables, that we employ in our SARIMA model to improve its forecast. But, to determine which one we would like to incorporate we have to make a correlation matrix to visualize which ones to pick. It looks like so:

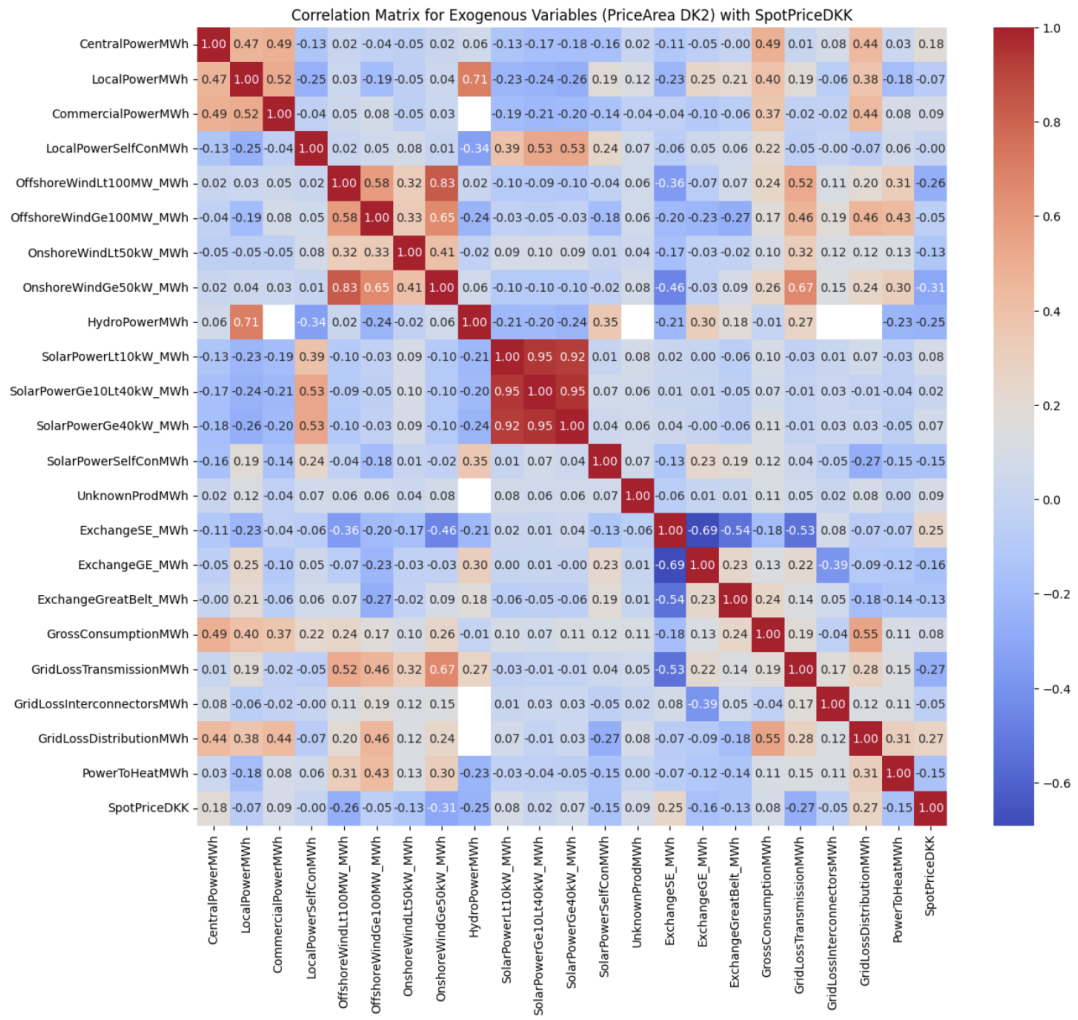


Figure 6: Correlation matrix for exogenous variables and spotprices for DK2

We can see that the exogenous variable with the highest correlation is "OnshoreWindGe50kW_MWh" with the correlation -0.306377. The correlation is low, but we would nonetheless like to see if our SARIMAX model can gain information about the data by looking at the exogenous variable too. We employ it in our model and end up with the spotprice plot:

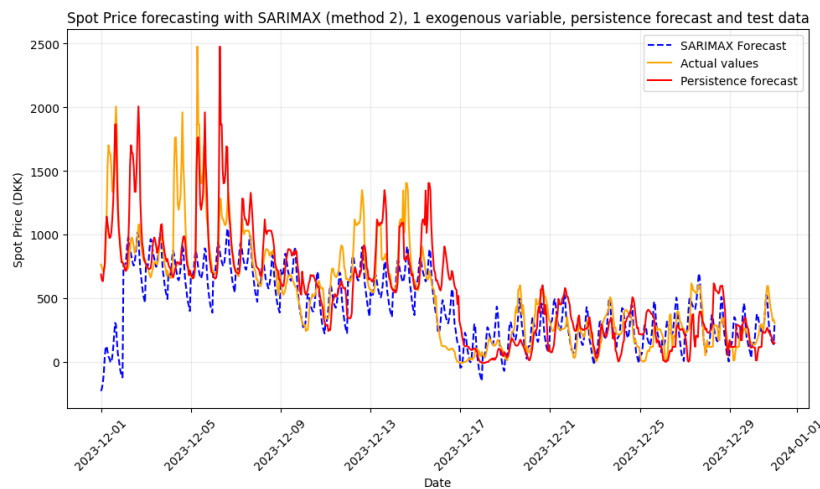


Figure 7: SARIMAX forecast (1 exogenous variable, method 2), persistence forecast, and testing data spotprice plot

The RMSE is 257 for daily persistence and 327 for the SARIMAX forecast. As we can see the SARIMAX model more closely forecasts the testing data on a day-ahead basis. This is due to the exogenous variable providing information about the future spotprice movements. We add the variable "GridLossDistributionMWh", and try again:

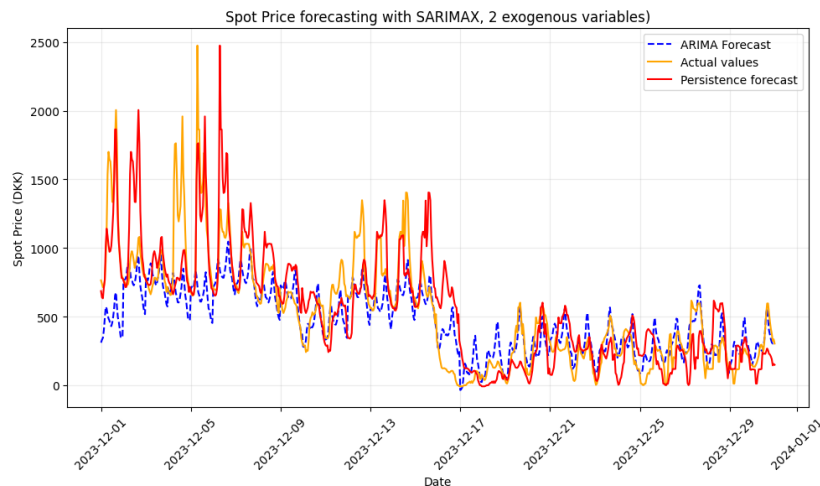


Figure 8: SARIMAX forecast (2 exogenous variables, method 2), persistence forecast, and testing data spotprice plot

RMSE for the SARIMAX forecast is 276.3498818042411. We see that the SARIMAX model with two exogenous variable even more closely forecasts the testing data than with only one exogenous variable. This is due to the SARIMAX model gaining information from two exogenous variables.

It is worth to point out how low the correlations are. Adding another exogenous variable might not increase the quality of the fit, and ultimately decrease the RMSE, as the correlations are so low. It is indeed important to note, that exogenous variables also have to be relevant to the target value. In this case, onshore wind power generation is highly relevant, and so might gridloss distribution also be. As it has to do with the amount of electricity available on the market at any given time, the clearer relationship being between onshore wind power generation and spotprices. We have proved our point that, broadly speaking, including exogenous variables in an SARIMA model increases the precision of the forecasts, as the model has more data to gain information from. The correlation has to be high, as the SARIMAX model is not an 'intelligent' model, i.e. the information that can be deducted is simply "if this parameter increases or decreases, then this parameter decreases or increases.

2 Task 2 - LSTM Model

In task 2, we aim to develop a Long Short-Term Memory (LSTM) network to predict the electricity prices. The reason was to try out a different model from the ARIMA to see if it would perform better forecasting.

2.1 Day-ahead prediction for the testing dataset (LSTM)

We've built the LSTM model to predict hourly electricity prices for the DK2 zone, using the same historical data as we did with the ARIMA (2019-2023). LSTMs are good at this kind of task because they can remember patterns over time, which is key when dealing with past data to predict the future. We again split our data into two parts: one for training the model and the other for testing it. The training set thus goes up to November 2023, and the test set includes December 2023. We used a MinMaxScaler to scale the prices, which helps the model learn more effectively (due to factors, that are outside the scope of this course). Our LSTM model has layers designed to remember and learn from the data, with dropout layers in between to prevent it from just memorizing the training set and failing to predict new data correctly. We also used an EarlyStopping callback, which stops training when the model stops getting better on the validation set. This helps make sure our model can work well on data it hasn't seen before.

After training for 20 epochs, the model's RMSE on the testing set was approximately **88.77**, while, as we now from earlier, the persistence model had a significantly higher RMSE of approximately **257**. After training, we then visualized the model's predictions against the actual prices for December 2023 combined with a persistence model forecast as seen below.

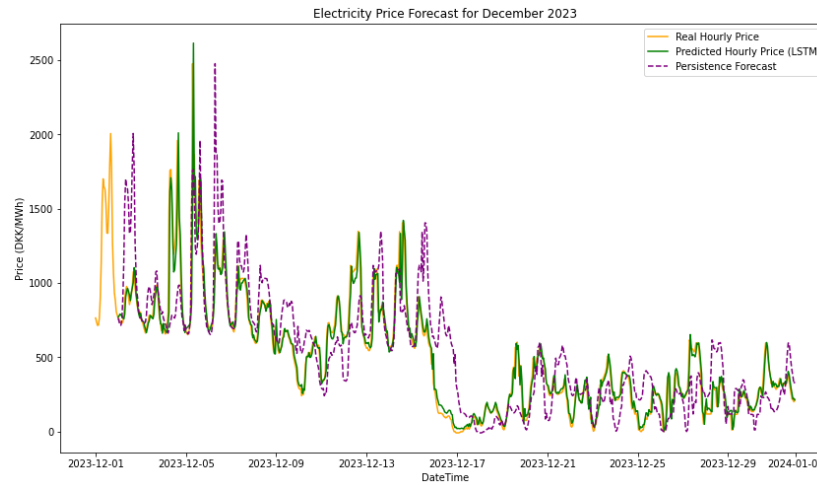


Figure 9: Actual vs. LSTM Forecast vs. Persistence Forecast

The resulting plot shows that the LSTM model closely follows the real hourly price trends for December 2023. In contrast, the persistence forecast diverges notably, indicating its lesser suitability for such predictions.

2.2 Day-ahead prediction including an exogenous variable (LSTMX)

In task 2.2, we incorporated the OnshoreWindGe50kW-MWh as an exogenous variable. This choice was driven by the observed correlation between wind energy production and electricity prices in the DK2 zone from our Correlation Matrix. By integrating this data, we aimed to improve the model's predictive accuracy. The model now processed both electricity prices and wind energy production, so we made adjustments in the input layer to handle the additional data. We continued using a MinMaxScaler, dropout layers for regularization, and an EarlyStopping callback to optimize training. We again, ran the model for 20 epochs. To our surprise, the new LSTMX model showed us a small worsening, achieving a RMSE of **93.27**, compared to the **89.42** RMSE from task 2.1 which used only historical prices. Since the RSME's are so similar, it is difficult to see the difference on the actual vs. forecast plot.

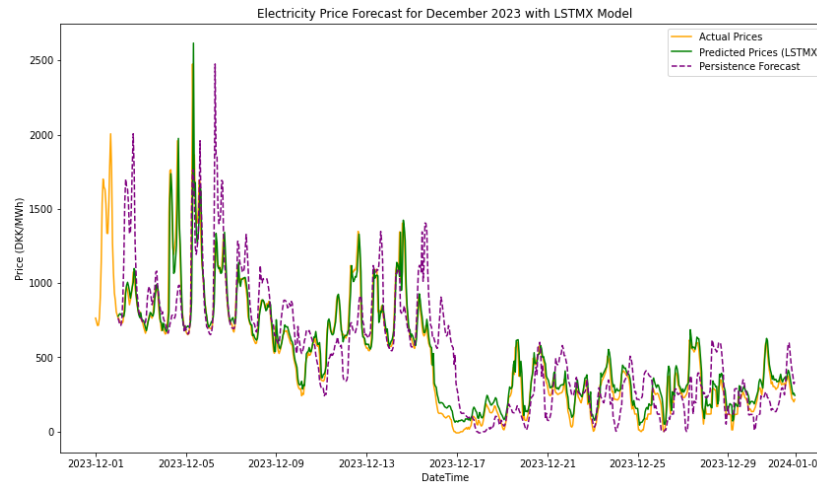


Figure 10: Actual vs. LSTM Forecast with Exogenous Variable vs. Persistence Forecast

Incorporating the exogenous OnshoreWindGe50kW-MWh variable did not enhance the model's performance, which contradicted to our expectations that the influence of wind energy has on electricity pricing. This does not make intuitive sense since today, over 40 percent of Denmark's electricity productions stems from wind turbines.¹ Though the correlation between spot price data and Onshore wind production was also found to be surprisingly low, c.f. correlation matrix from earlier. Moving on, when looking at the training process, the LSTM and LSTMX models both saw a decreasing trend in loss over epochs, suggesting improvement in learning. However, already after 2 epochs the improvement seems insignificant, which could indicate that our models have a very steep learning curve. Knowing this, we could have run the training for fewer epochs and thereby achieve an highly improved training time.

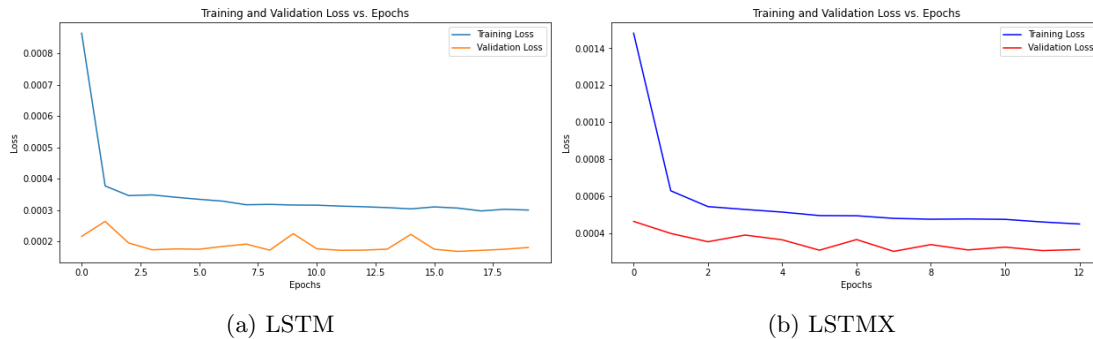


Figure 11: Training and validation loss vs. Epochs

¹<https://ens.dk/ansvarsomraader/vindmoeller-paa-hav/fakta-om-vindenergi-paa-hav>

3 Task 3 - Comparing the performance of the ARIMA and LSTM models

In this task we compare the ARIMA and LSTM models.

3.1 Comparing the performance of the SARIMAX and LSTMX models

We have made the following table showcasing the RMSE values of the different models we have created in this assignment:

Table 1: Comparing Accuracy (RMSE) and Training Time (t)

Model	RMSE	t
Daily Persistence	257	< 1s
SARIMA Forecast (method 1)	232	749.85s
SARIMA Forecast (method 2)	327	70.04s
SARIMAX Forecast (method 2)	276	77.58s
LSTM Forecast	89.42	573s
LSTMX Forecast	93.27	573s

In task 1 we established the fact that forecast we create with method 1 is better than the forecasts we create with method 2, also including exogenous variable. This is due to the fact that we only let the automatic arima function run for a certain number of iterations to save time, and with out manual model we had a more focused exploration of the parameters. The exogenous variable increased the performance, but it was terrible compared to the long short term memory model and LSTMX model, that achieve 1/3 and 1/4 of the RMSE respectively.

The reason for the difference in performance between the ARIMA and LSTM model lies in the nature of the models. The ARIMA models The SARIMAX model assumes linear relationships between past observations, forecast errors, and exogenous variables. This model captures seasonal patterns and trends into account. Again, the linear relationships. The LSTMX model is based on deep learning principles and captures complex nonlinear relationships and long-term dependencies into account, including effects of exogenous variables. So, the SARIMAX model is limited by linear relationships, and the LSTMX model can model non-linear relationships. The world is not simple, especially not something like spotprice data. So, a model that can take

more of the complexity into account is better. Therefore, the LSTMX model is better.

The LSTMX model is computationally more expensive to run than the SARIMAX model. The training time for the SARIMAX model was 78 seconds, while the training time for the LSTMX forecast was 1303 seconds. The SARIMAX model is involved estimating parameters for autoregressive, moving average, and exogenous components. It requires iterative methods such as maximum likelihood estimation. Simple models do not require too much computing power, though this increases drastically as our model becomes more complex (higher values of p , q , and seasonal parameters too). We only use two months of the training data to train our SARIMAX model. The LSTMX model is a deep learning model based on a neural network. It is trained using the optimization algorithm Adam. So, training the LSTMX involves computationally expensive operations, especially when dealing with large datasets. The intensive operations ultimately mean we get a much higher training time compared to the SARIMAX model.

The SARIMAX model had more parameters and was harder to train than the LSTMX model. In the SARIMAX model we had 8 parameters to tune: $\text{ARIMA}(p,d,q)(P,D,Q)[m]$, and the choice of which exogenous variable to chose (and even how many!). Furthermore, there was the consideration of which data to pick for the training. If one had the time one would let the `auto.arima` function run for hours on end to find the best parameters for the model, balancing effectivity and accuracy. With the LSTMX model we only had the opportunity to pick the exogenous variable to train the model. Otherwise, the neural networks and mechanisms of LSTMX handled the rest for us.

3.2 Recommendation

To make a recommendation of which model to use for electricity price forecasting we take training time, efficiency, and precision into account.

Throughout task 1 and 2 we have gained knowledge about the performance of our SARIMAX and LSTMX model in prediction of electricity spotprices. We have found, that the most accurate model was, by far, the LSTMX model (compared to the SARIMAX model). The model takes far longer to train, though. Considering our recommendation is for day-ahead predictions, and not 10-minute ahead predictions, the long training time does not matter. The higher accuracy far outweighs the higher computational intensity and training time.

So, a clear recommendation from our engineering company is to use the LSTMX model for day-ahead predictions.

In conclusion, the LSTMX model significantly outperforms the SARIMAX model in terms of

accuracy. This is due to the ability to capture complex nonlinear relationships and long-term dependencies in the data. The difference in performance between the models is attributed their underlying principles. The LSTMX model is computationally expensive, and takes far longer to train. Despite the cons, the clear recommendation in picking the best model is the LSTMX model.

4 Task 4 - Total profits for December 2023

For the example scenario, we used the below specifications for the battery system:

- The battery's state of charge can vary between 10% and 100%.
- The battery has a 50% SOC at the start of the simulation (on December 1st).
- The battery has a power capacity of 1 MW and an energy capacity of 2 MWh.
- Charging and discharging efficiency is equal to 95%, i.e., $\eta_c = \eta_d = 0.95$.
- Each day the schedule of the battery is optimized between 00:00 and for the next 24 hours.
- Energy is bought and sold at the spot price (not considering any taxes or tariffs).
- The constraint of the final or end SOC, i.e., SOC at the end of the day, should be set to 50%. We set it to 50% to make life easier for ourselves code-wise. But, also to allow the most freedom energy-wise for the beginning of each day. I.e. when we start a new day we would like to be able to sell / buy an equal amount as we don't know which direction the spotprices will take. One could empirically test the result of changing the state of charge at the end of day. Most likely, the SOC will be higher or lower, depending on daily tendencies like low prices in the beginning of the day (so, low SOC at the end of the day, ready to purchase cheap energy). But, we have not examined this, and made life easier simply by picking 50% SOC.

With the above mentioned scenario and parameters, we can set up a program (in Python), using code from Assignment 1 (DTU course 46040, code author: Professor Ziras). Here among the code for the optimization of a battery and an alteration of the load function. As previously stated, the whole code will be attached at the hand-in of this report for review and use. Note that for this investigation the LSTM forecast model was used, which was deemed the most precise, compared to the ARIMA model, as can be seen from the results in the previous tasks. To compare the potential profits of December 2023, using the various channels to optimize our battery, we set up a bar chart as can be seen in Figure 12:

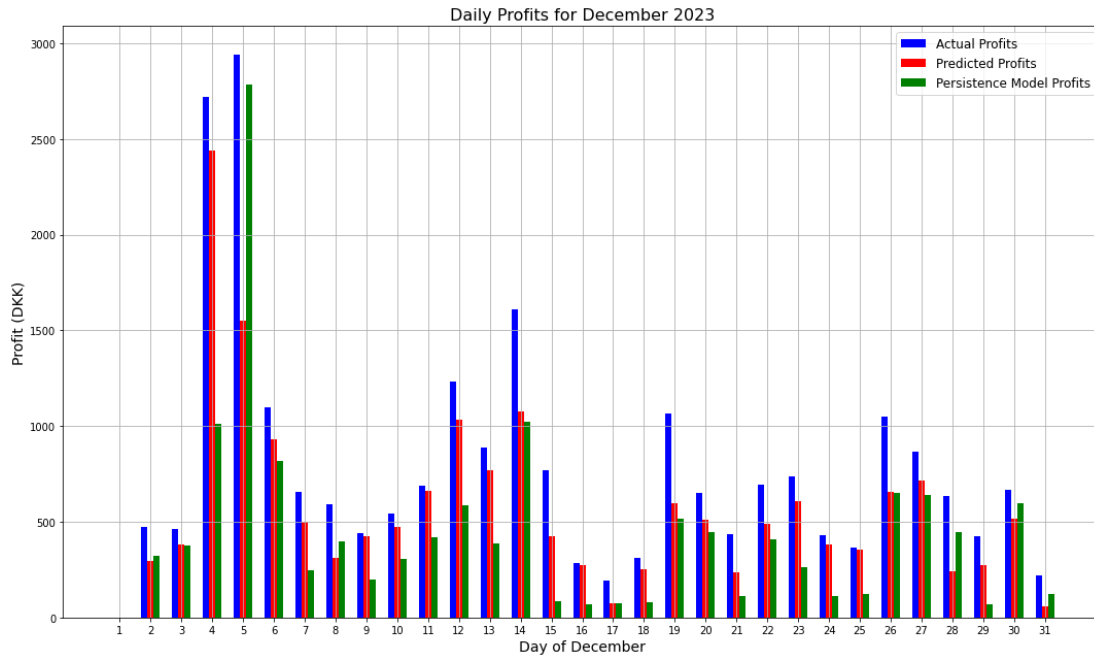


Figure 12: The comparison of daily profits in December 2023, using various knowledge or models to optimize the battery

Resulting in a total profit for December of:

- With perfect price knowledge, the owner could obtain a monthly profit of: 24.144 DKK
- Predicted monthly profit using LSTM: 17.515 DKK
- Persistence model monthly profit: 13.701 DKK

In the following section, we will try and compare the two different forecasting methods with the actual price to see how well or bad they performed and why.

4.1 Comparing potential profit scenarios, either by LSTM forecast or by perfect price knowledge

Comparing the profits achieved with the LSTM's forecasted spot prices against the scenario of having perfect price knowledge of December, we see that the prediction is notable lower than the profits achieved with perfect price knowledge by 27%. That may not denote as a significant deviation considering this being a forecast versus reality, but it is still noteworthy in the context of this battery being a business investment, that the owner will be off by this much.

In all days recorded the total daily profit from using perfect price knowledge exceeds that of the predicted profit. That makes sense since modelling with the actual spot prices would make the

optimization ideal.

4.2 Investigating using persistence model prices to optimize battery the day ahead

When using the persistence model prices to optimize the battery, results deviate greater from perfect price knowledge than with the predicted prices. This was expected, since simply by intuition there comes a greater uncertainty of basing ones forecast solely on what happened yesterday. This is the results that is most practical to compare, since using a persistence model might be one of the more simple and usual alternatives for a battery owner to do any kind of day-ahead prediction - especially back in the earlier days.

The added value of the prediction, compared to simply using persistence is pretty clear: 28% higher profits. Again, using the context of this being an investment or business, improving revenue by 28% from simply altering the method or software used to predict, is an interesting observation. Only at four instances did the persistence model achieve better results, respectively at the 5th, 28th, 30th and 31st of December. This might be due to unexpected circumstances or factors, that made a factor-predicting forecast worse than the persistence model, which in some sense, are a more random method, when only based on the results the day before and not more intricately taking in both short ad long term knowledge as in the case of the LSTM.

4.3 Key features to deliver a good profit-maximizing bidding strategy

Key features to a good profit-maximizing bidding strategy, is at the basis, two things: a good prediction model and a good optimization tool. And lastly the battery itself is of course a factor, but here we do not consider this as part of the strategy.

The key for the prediction model is to determine the best model and important factors (i.e. exogenous) which perhaps should be included, to best forecast the day-ahead and further.

One key for the optimization tool to determine bidding, is to decide or optimize for the SOC constraint's. To make sure the battery is always considering ahead, e.g. in a scenario of cheap energy later in the day followed by a high sell price, means the battery at this point in time should keep a low SOC, to be able to bid and store this energy up later. Therefore ideally the N.SOC should constantly be reconsidered and optimized for. That is some features we could identify to optimize profit-maximizing.

5 Lessons learned

1. Careful Coding: We learned that choosing the right settings for our models is very important. Next time, we'll spend more time testing different settings to find the best ones. It's good to mix automatic tools with our own checks to get the best results.

2. Choosing the Right Model: This project showed us that some models can handle complex data better than others. For example, the LSTM models did much better than the simpler ARIMA models because they can understand complex patterns. This teaches us to pick our models carefully based on what our data is like.

3. Using Additional Data Carefully: Adding extra data, like wind energy output, didn't always help as much as we hoped. We need to make sure the extra data we use has a strong connection to what we're trying to predict. This will help us avoid making our models too complicated without improving our results.

6 Conclusion

This report looked at how well ARIMA and LSTM models can predict electricity prices in Denmark's DK2 area. We started with the ARIMA model, which got an RMSE of 232. When we added more information to the model (SARIMAX), the RMSE went up to 327. However, the LSTM models did much better, with an RMSE of 89.42 without extra data, and an slightly worse 93.27 when we added more information.

The LSTM models were more accurate because they could understand complex patterns and remember important details from the past, which is crucial for predicting prices correctly. These models were very useful when we combined them with strategies for managing battery storage, helping to make better trading decisions and increase profits.

In conclusion, the LSTM models, especially when we added extra data to them, were the most effective way to predict electricity prices. This shows that using advanced machine learning methods can really help in the energy market, leading to smarter and more profitable decisions.